Homework 09

Author: Manushri Pendekanti (mp79)

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline
```

Question 1: Choosing an SVM kernel

Use `data/saheart.csv`. Predict `chd` with the same nine predictors as Homework 7; code `famhist` as 1 = Absent, 2 = Present.

Compare linear, polynomial (degrees 2–4), and RBF SVMs. Use **no more than 20 candidate settings total**. Use the **same five folds** for all candidates; standardize predictors within each fold using training-fold data only.

In [2]:
```
from sklearn.model_selection import StratifiedKFold
from sklearn.svm import SVC

heart = pd.read_csv("data/saheart.csv")
heart["famhist"] = heart["famhist"].map({"Absent": 1, "Present": 2})

predictors = [
    "sbp", "tobacco", "ldl", "adiposity", "famhist",
    "typea", "obesity", "alcohol", "age"
]

X = heart[predictors].to_numpy()
y = heart["chd"].to_numpy()


folds = list(
    StratifiedKFold(n_splits=5, shuffle=True, random_state=1).split(X, y)
)

candidates = []


for C in [0.1, 1, 10]:
    candidates.append(("linear", None, C, None))


for degree in [2, 3, 4]:
    for C in [0.1, 1]:
        candidates.append(("polynomial", degree, C, None))
```

```
for gamma in [0.01, 0.1, 1]:
    for C in [0.1, 1, 10]:
        candidates.append(("radial", None, C, gamma))

results = []

for kernel, degree, C, gamma in candidates:
    fold_errors = []

    for train_index, valid_index in folds:
        # Standardize using only the fitting observations.
        center = X[train_index].mean(axis=0)
        scale = X[train_index].std(axis=0, ddof=1)

        X_train = (X[train_index] - center) / scale
        X_valid = (X[valid_index] - center) / scale

        if kernel == "linear":
            fit = SVC(kernel="linear", C=C, tol=1e-7)
        elif kernel == "polynomial":
            fit = SVC(
                kernel="poly",
                degree=degree,
                C=C,
                gamma=1 / len(predictors),
                coef0=1,
                tol=1e-7
            )
        else:
            fit = SVC(kernel="rbf", C=C, gamma=gamma, tol=1e-7)

        fit.fit(X_train, y[train_index])
        predictions = fit.predict(X_valid)
        fold_errors.append(np.mean(predictions != y[valid_index]))

    results.append({
        "kernel": kernel,
        "degree": degree,
        "C": C,
        "gamma": gamma,
        "mean_error": np.mean(fold_errors)
    })

for result in results:
    print(
        "Kernel: {result['kernel']}, "
        "degree: {result['degree']}, "
        "C: {result['C']}, "
        "gamma: {result['gamma']}, "
        "mean CV error: {result['mean_error']:.4f}"
    )

# i held gamma = 1/9 and coef = 1 for polynomial kernels constant
# I would choose the quadratic poly kernel where C = 1
#because with a lot of tied with toehr for mean error but had smallest degre
```

```
Kernel: {result['kernel']}, degree: {result['degree']}, C: {result['C']}, ga
mma: {result['gamma']}, mean CV error: {result['mean_error']:.4f}
Kernel: {result['kernel']}, degree: {result['degree']}, C: {result['C']}, ga
mma: {result['gamma']}, mean CV error: {result['mean_error']:.4f}
Kernel: {result['kernel']}, degree: {result['degree']}, C: {result['C']}, ga
mma: {result['gamma']}, mean CV error: {result['mean_error']:.4f}
Kernel: {result['kernel']}, degree: {result['degree']}, C: {result['C']}, ga
mma: {result['gamma']}, mean CV error: {result['mean_error']:.4f}
Kernel: {result['kernel']}, degree: {result['degree']}, C: {result['C']}, ga
mma: {result['gamma']}, mean CV error: {result['mean_error']:.4f}
Kernel: {result['kernel']}, degree: {result['degree']}, C: {result['C']}, ga
mma: {result['gamma']}, mean CV error: {result['mean_error']:.4f}
Kernel: {result['kernel']}, degree: {result['degree']}, C: {result['C']}, ga
mma: {result['gamma']}, mean CV error: {result['mean_error']:.4f}
Kernel: {result['kernel']}, degree: {result['degree']}, C: {result['C']}, ga
mma: {result['gamma']}, mean CV error: {result['mean_error']:.4f}
Kernel: {result['kernel']}, degree: {result['degree']}, C: {result['C']}, ga
mma: {result['gamma']}, mean CV error: {result['mean_error']:.4f}
Kernel: {result['kernel']}, degree: {result['degree']}, C: {result['C']}, ga
mma: {result['gamma']}, mean CV error: {result['mean_error']:.4f}
Kernel: {result['kernel']}, degree: {result['degree']}, C: {result['C']}, ga
mma: {result['gamma']}, mean CV error: {result['mean_error']:.4f}
Kernel: {result['kernel']}, degree: {result['degree']}, C: {result['C']}, ga
mma: {result['gamma']}, mean CV error: {result['mean_error']:.4f}
Kernel: {result['kernel']}, degree: {result['degree']}, C: {result['C']}, ga
mma: {result['gamma']}, mean CV error: {result['mean_error']:.4f}
Kernel: {result['kernel']}, degree: {result['degree']}, C: {result['C']}, ga
mma: {result['gamma']}, mean CV error: {result['mean_error']:.4f}
Kernel: {result['kernel']}, degree: {result['degree']}, C: {result['C']}, ga
mma: {result['gamma']}, mean CV error: {result['mean_error']:.4f}
Kernel: {result['kernel']}, degree: {result['degree']}, C: {result['C']}, ga
mma: {result['gamma']}, mean CV error: {result['mean_error']:.4f}
```

In [ ]:

Question 2: Write and use a short SVM analysis skill

Write `svm-analysis/SKILL.md` **by hand** (≤50 lines after frontmatter; no dataset-specific names in the skill body).

Explicitly invoke the skill on `data/saheart.csv` with the same response, predictors, and `famhist` coding as Question 1. Include a **one-page 11-point** analysis report in this submission and append the full `SKILL.md` after it.

Question 2 — Part a

Skill submission

Question 2 — Part b

Complete skill file (`skill/skill.md`):

```
---
name : SVM Model Fitting Skill
description : Run SVM Model Fitting and then make a table
explaining choices and results and return one page report of font
11 size and it is only invoked when a user requests the use of this
skill
month: october
---

## Step to follow to construct report

Make sure to follow this order exactly and don't assume in any
situation and do your best to pull from the dataset and given data
from user and ALWAYS follow the steps properly and NEVER miss any
parts of this steps

1. First you are going to take the given dataset and review the
dataset to see if it works for a SVM model fitting including
thinking of things like if data is either seperable linear or non-
linear, data is in numerical vectors and each response has neat
structured predicators value, etc

2. Next you should consider different kernels including linear,
polynomial(2,3,4) degrees, and radical basis and the linear help
with linearly seperable one and other one help with those not
linear


3. In terms of parameters to be tuned like cost C, gamma, etc based
on the specfic kernel and any other params which you would believe
would be important and use method  cross validation to tune the
parameters

4. For evaulation method use cross validation mean error, a ROC
curve, or mix of these for better results which helps provide
evidence for the final model

5. Make sure to include a good graph and summary table to showcase
the justification behind the choices and descisions you made

6. interprut the fit through the what different tuning params or
settings do, the important information learnt about the boundr it
picked based on data, etc and finally pick out the important
numerical results based on values that help justify or showcase the
model's accuracy and what it does well and summary or snapshot
```

```
## Output

Make sure to write all the results gathered from the above steps
into a 1 page report of font size 11 and make it detailed but still
keep it compact enough that person can get a clean summary

Make sure to format it well so it easy to follow and understand for
anyone

## Important things to remember

NEVER exceed 1 page
dont look at test set when training
```

Question 2 — Part c

One-page analysis report (11 pt) — skill invoked on `data/saheart.csv` (response `chd`; nine predictors; `famhist`: Absent→1, Present→2).

Data suitability. The data have **n = 462** observations and **p = 9** numeric predictors in fixed-length vectors, with binary response CHD (0: 302, 1: 160). There are no missing values after coding `famhist`. The classes are imbalanced but both are represented, so SVM fitting is feasible. The relationship is unlikely to be perfectly linear in all nine variables, so linear and flexible kernels were compared. Predictors were standardized within each cross-validation fold using training-fold statistics only.

Kernels and tuning. We compared **linear**, **polynomial** (degrees 2, 3, 4; `coef0 = 1`), and **RBF** SVMs (`sklearn.svm.SVC`). **Cost C** was tuned over {0.1, 1, 10}; for RBF, **γ** over {0.01, 0.1, 1}. **20 candidate settings** total. A **70/30 stratified holdout** (seed **43207**; 323 train, 139 test) was fixed first; the test set was not used for tuning. **Five stratified CV folds** (same folds for all candidates) on the training set scored each setting by **mean misclassification rate** (±1 SE). Out-of-fold predicted probabilities for the selected model gave a **ROC curve** (training CV only).

Results (best setting per kernel family).

Kernel family	Best setting	Mean CV error	SE
Linear	C = 0.1	0.282	0.022
Quadratic poly	C = 0.1	**0.272**	**0.006**
Cubic poly	C = 0.1	0.275	0.017
Quartic poly	C = 0.1	0.297	0.014
RBF	C = 10, γ = 0.01	0.273	0.011

Always predicting the majority class gives error **0.346** (dotted reference in the figure).

Selected model. Quadratic polynomial SVM, C = 0.1 (lowest mean CV error). RBF (0.273) is within about **1 SE**, so evidence for a unique winner is weak; the quadratic kernel gives a modest extension beyond linear boundaries without the highest-degree polynomials, which overfit (degree 4 CV ≈ 0.297). Small **C** penalizes margin violations and reduces overfitting on this clinical data.

Test evaluation (single holdout). Test misclassification rate = **0.288** (40/139). Confusion matrix (rows = true, cols = predicted):

	Pred 0	Pred 1
True 0	81	10
True 1	30	18

Training out-of-fold **ROC AUC ≈ 0.746** for the selected model. Test error is close to CV (0.272), with more false negatives (30) than false positives (10) for CHD.

Figure. Bar chart of the ten lowest-CV settings vs. majority baseline; ROC from out-of-fold training predictions for the selected model (next cell).

Limitations. One random split; SVMs require comparable feature scales (addressed by fold-wise scaling); boundaries are hard to visualize in nine dimensions; misclassification rate treats both classes equally despite imbalance.

Python 3.13; scikit-learn 1.8.0; seed 43207.

In [3]:
```
from sklearn.model_selection import train_test_split, StratifiedKFold, cross
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.svm import SVC
from sklearn.metrics import confusion_matrix, roc_curve, auc

SEED = 43207
predictors = [
    "sbp", "tobacco", "ldl", "adiposity", "famhist",
    "typea", "obesity", "alcohol", "age"
]
heart = pd.read_csv("data/saheart.csv")
heart["famhist"] = heart["famhist"].map({"Absent": 1, "Present": 2})
X = heart[predictors].astype(float)
y = heart["chd"].to_numpy()
majority_err = 1 - pd.Series(y).value_counts().max() / len(y)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=SEED, stratify=y
)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)

def cv_misclass(clf, X_tr, y_tr):
    errs = []
    for tr, va in cv.split(X_tr, y_tr):
```

```
        clf.fit(X_tr.iloc[tr], y_tr[tr])
        errs.append(np.mean(clf.predict(X_tr.iloc[va]) != y_tr[va]))
    arr = np.array(errs)
    return arr.mean(), arr.std(ddof=1) / np.sqrt(len(arr))

Cs = [0.1, 1, 10]
gammas = [0.01, 0.1, 1]
candidates = []
for C in Cs:
    candidates.append((f"linear, C={C}", {"C": C, "kernel": "linear"}))
for deg in [2, 3, 4]:
    for C in [0.1, 1, 10][:2 if deg == 4 else 3]:
        candidates.append((f"poly d={deg}, C={C}", {"C": C, "kernel": "poly"
for C in Cs:
    for g in gammas:
        candidates.append((f"RBF, C={C}, γ={g}", {"C": C, "kernel": "rbf", "
candidates = candidates[:20]

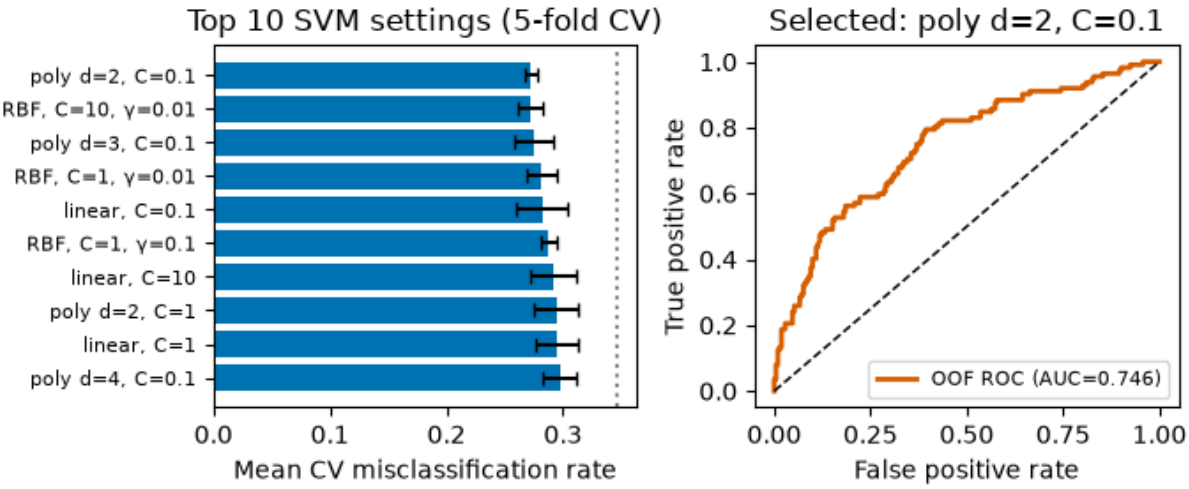
rows = []
for label, params in candidates:
    pipe = Pipeline([
        ("scale", StandardScaler()),
        ("svc", SVC(**params, probability=True, random_state=SEED)),
    ])
    m, se = cv_misclass(pipe, X_train, y_train)
    rows.append((label, m, se, params))
rows.sort(key=lambda t: t[1])
best_label, best_cv, best_se, best_params = rows[0]

best_pipe = Pipeline([
    ("scale", StandardScaler()),
    ("svc", SVC(**best_params, probability=True, random_state=SEED)),
])
best_pipe.fit(X_train, y_train)
test_err = np.mean(best_pipe.predict(X_test) != y_test)
cm = confusion_matrix(y_test, best_pipe.predict(X_test))
proba_oof = cross_val_predict(best_pipe, X_train, y_train, cv=cv, method="pr
fpr, tpr, _ = roc_curve(y_train, proba_oof)
roc_auc = auc(fpr, tpr)

plot_rows = rows[:10]
fig, axes = plt.subplots(1, 2, figsize=(7, 3))
ax = axes[0]
labels = [r[0] for r in plot_rows]
means = [r[1] for r in plot_rows]
ses = [r[2] for r in plot_rows]
ypos = np.arange(len(labels))
ax.barh(ypos, means, xerr=ses, capsize=3, color="#0072B2")
ax.set_yticks(ypos)
ax.set_yticklabels(labels, fontsize=8)
ax.axvline(majority_err, color="gray", ls=":", label="Majority class")
ax.set_xlabel("Mean CV misclassification rate")
ax.set_title("Top 10 SVM settings (5-fold CV)")
ax.invert_yaxis()
ax = axes[1]
ax.plot(fpr, tpr, color="#D55E00", lw=2, label=f"OOF ROC (AUC={roc_auc:.3f})
```

```
ax.plot([0, 1], [0, 1], "k--", lw=1)
ax.set_xlabel("False positive rate")
ax.set_ylabel("True positive rate")
ax.set_title(f"Selected: {best_label}")
ax.legend(fontsize=8, loc="lower right")
fig.tight_layout()
fig.savefig("q2-svm-report-figure.png", dpi=150, bbox_inches="tight")
plt.show()

print("Selected:", best_label)
print(f"CV error: {best_cv:.3f} (SE {best_se:.3f})")
print(f"Test error: {test_err:.3f}")
print("Confusion matrix:\n", cm)
```




```
Selected: poly d=2, C=0.1
CV error: 0.272 (SE 0.006)
Test error: 0.288
Confusion matrix:
 [[81 10]
 [30 18]]
```